III. PROPOSED METHODOLOGY

A. System Overview

Academic advising in engineering colleges has long suffered from a fundamental mismatch: the problems students face are deeply personal, context-dependent, and time-sensitive, yet the guidance available to them is generic, periodic, and reactive. A student struggling with a 58% internal mark in Artificial Intelligence while simultaneously managing 68% attendance in the same subject needs a very different response than one who has strong attendance but poor marks — yet most existing systems would offer both the same boilerplate advice.

The system proposed in this work, the Intent-Aware Dynamic Multi-Agent Framework for Personalized Academic Recommendations (IDMAF-PAR), is designed around the premise that meaningful academic guidance requires three things working together: a precise understanding of what the student is actually asking, a rich representation of their current academic state, and multiple specialized reasoning perspectives acting in concert. None of these alone is sufficient.

The architecture reflects this philosophy through four tightly coupled layers. The conversational interface, built in React, accepts natural language queries from students — the same way they would ask a trusted senior or faculty advisor. The Node.js Fastify backend serves as the orchestration center: it routes queries to the right combination of agents, runs them in parallel, resolves conflicts between their outputs, ranks recommendations by confidence, and constructs an explainable final response grounded in the student's own data. A dedicated Python FastAPI microservice handles all machine learning inference — intent classification, embedding generation, and confidence scoring — cleanly separated from the orchestration logic so that models can be upgraded independently. Finally, a dual data layer keeps structured academic records in PostgreSQL via Prisma ORM and indexes unstructured learning content using pgvector for semantic retrieval.

What distinguishes this system from prior work is not any single component in isolation, but the pipeline that connects them: intent detection feeds dynamic agent selection, parallel execution feeds evidence-aware fusion, and fusion feeds explainable confidence-ranked output. Each stage is designed to make the next stage's job more precise.

B. Student State Representation

Before any recommendation can be generated, the system must know who it is talking to. This sounds obvious, but most academic AI systems treat student context as an afterthought — a few fields appended to a prompt. In IDMAF-PAR, the student's academic state is a first-class citizen of the system, formally defined and actively used at every stage of the pipeline.

Each student is represented as a multi-dimensional state vector S:

$$S = \{ P, A, M, H, E, R \}$$

where P captures stable identity — department, semester, roll number. A is an attendance map keyed by subject, recording the current attendance percentage for each enrolled course. M captures internal assessment performance: marks, maximum marks, and assessment type per subject. H holds historical performance data across previous semesters, enabling the system to detect trends — whether a student is improving, declining, or stagnant in a given domain. E encodes the examination schedule, giving the system awareness of urgency: a recommendation for a subject with 32 days remaining is structurally different from one with 7 days remaining. Finally, R retains the history of previous recommendations and queries, so the system does not give the same advice repeatedly or ignore what has already been tried.

The structured components — P, A, M, H, E — live in PostgreSQL. Unstructured learning content associated with each subject (lecture notes, reference topics, common weak areas) is encoded using the sentence-transformers/all-MiniLM-L6-v2 model and stored as vector embeddings in pgvector, enabling semantic retrieval at query time. When a student submits a query, the Context Engine assembles the complete state S in a single operation — joining relational records and retrieving the top-k most semantically relevant content vectors — and hands this unified context to the router. No agent ever operates on incomplete or stale data.

C. Academic Decision Identification

The challenge of understanding what a student is actually asking is more subtle than it first appears. A message like *"macha, my AI subject is killing me, internal also bad and attendance also less, how do I fix this before semester exam"* carries at least four distinct intents simultaneously — performance analysis, attendance concern, study planning, and exam preparation — none of

which is explicitly stated as a label. A system that can only detect one intent per query will systematically underserve students with complex, multi-faceted problems.

The Intent Detection Layer addresses this through structured multi-label classification. Implemented as a Python FastAPI microservice, it uses a fine-tuned BERT model (bert-base-uncased) trained on a curated dataset of student academic queries labeled across ten intent categories:

```
ACADEMIC_QUERY · ATTENDANCE_QUERY · MARK_ANALYSIS
STUDY_PLAN · CAREER_GUIDANCE · ROADMAP_REQUEST
PERFORMANCE_ANALYSIS · RESOURCE_RECOMMENDATION
EXAM_PREPARATION · GENERAL_QUERY
```

The output of the classifier is a structured intent object rather than a free-form string:

```
IntentOutput {
  primary_intent    : ENUM
  secondary_intents : List[ENUM]
  confidence        : Float ∈ [0, 1]
  embeddings        : Float[384]
}
```

The decision to use a structured classifier rather than prompting a large language model for intent labels was deliberate. LLM-generated intent labels are non-deterministic — the same query can produce different label sets across runs, making evaluation unreliable and downstream routing unstable. A classification head with a fixed label space produces reproducible outputs, supports standard evaluation metrics (precision, recall, F1 per intent class), and feeds a deterministic signal to the agent router.

A zero-shot BART model (facebook/bart-large-mnli) serves as the baseline against which the fine-tuned BERT classifier is benchmarked in the evaluation section. The comparison between these two approaches — and the magnitude of improvement from fine-tuning on domain-specific student query data — constitutes one of the quantitative contributions of this work.

D. Problem Decomposition and Reasoning Capability Identification

Once the system knows what the student is asking, it faces a second question: which kinds of reasoning are actually needed to answer it well? This is the problem decomposition stage, and it is where most existing multi-agent systems make their first significant error — by activating every available agent for every query, regardless of relevance.

The cost of unnecessary agent activation is real. It increases latency, introduces noise into the fusion stage, and produces recommendations that feel unfocused because they blend perspectives that were never relevant to the query in the first place. A student asking about career guidance does not benefit from an attendance analysis agent weighing in on their question.

IDMAF-PAR maps each agent to the specific reasoning capability it provides and the student state fields it requires:

AgentReasoning CapabilityRequired State Fields		
Learning Agent	Concept gaps, study strategies, weak topic identification	M, H, subject content vectors
Performance Agent	Attendance and mark trend analysis, subject-wise risk profiling	A, M, H
Planning Agent	Daily, weekly, and exam-oriented schedule generation	E, M, A
Risk Agent	Academic risk classification across Low / Medium / High tiers	A, M, H
Resource Agent	Course, video, and reference material recommendations	M, subject content vectors
Roadmap Agent	Semester-level, skill-level, and placement learning paths	P, H, career context
Career Agent	Placement readiness assessment, skills gap identification	P, H, R

This capability map is encoded as a binary matrix $C \in \{0,1\}^{(|agents| \times |intents|)}$ where $C[a][i] = 1$ if agent a carries reasoning relevant to intent i. Combined with a learned relevance score that accounts for the richness of available student state, this matrix drives the selection of the minimal sufficient agent team for each query — enough agents to cover all detected intents, no more.

E. Problem-Specific Agent-Team Construction

The Dynamic Router takes the intent output and the capability matrix as inputs and constructs a query-specific agent team through a three-stage hybrid selection process. The hybrid design is intentional: pure rule-based selection is too rigid for the natural variability of student language, while pure LLM-based selection is too unpredictable for a system that needs to be evaluated and explained.

Stage 1 — Rule-based filtering eliminates agents that are structurally irrelevant to the detected intents. If no career-related intent is detected, the Career Agent is excluded before any scoring takes place. These rules are hard constraints, not soft preferences, and they ensure that the router never activates an agent whose reasoning has no connection to what the student asked.

Stage 2 — Relevance scoring ranks the remaining candidate agents using a weighted combination of intent alignment and context availability:

$$score(a, q) = \alpha \cdot C[a][primary_intent] + \beta \cdot \Sigma\, C[a][i] \text{ for } i \in secondary_intents + \gamma \cdot contextRelevance(a, S)$$

The *contextRelevance* term penalizes agents whose required state fields are missing or sparse for the current student — for example, if a student has no historical semester data, the agents that depend heavily on H receive a lower score even if their intent alignment is strong. This prevents the system from activating agents that would produce low-quality outputs due to data insufficiency.

Stage 3 — LLM arbitration handles the genuinely ambiguous cases — queries where the score stage produces near-tied candidates or where the phrasing is unusual enough that the rule and score stages together do not produce a high-confidence selection. A single, constrained LLM call resolves these ties. Crucially, the LLM is not generating the recommendation here — it is making a binary activation decision for a bounded set of candidate agents, which keeps its role narrow and its output verifiable.

The confirmed agent team T is passed to the parallel execution engine.

F. Parallel Agent Reasoning

With the agent team confirmed, the system executes all selected agents simultaneously. This is implemented using Promise.all in the Node.js orchestration layer:

javascript

```
const agentOutputs = await Promise.all(
  selectedAgents.map(agent => agent.run(studentContext))
);
```

Each agent is a stateless async function that receives the full student context object and returns a standardized output:

typescript

```
AgentOutput {
  agentId        : string
  recommendation : string
  evidence       : Evidence[]
  reasoning      : string
  confidence     : Float
  metadata       : Record<string, any>
}
```

The stateless design is worth emphasizing. Agents do not communicate with each other during execution, do not share any mutable runtime state, and do not depend on the output of other agents to produce their own. This is intentional — it prevents cascading errors, makes individual agent outputs independently auditable, and means that any single agent failure does not corrupt the outputs of others. It also enables precise latency measurement per agent, which is reported in the evaluation.

From a systems perspective, the total response time for a query is bounded by the slowest agent in T rather than the sum of all agent latencies. For a typical query activating four agents with individual latencies of 400ms, 380ms, 450ms, and 410ms,

sequential execution would take approximately 1640ms. Parallel execution completes in approximately 450ms — a speedup factor approaching $|T|$ for agents with similar individual latencies.

G. Evidence Verification and Conflict Resolution

Parallel execution produces multiple independent agent outputs, and this is where a naïve system would simply concatenate them and call it a recommendation. IDMAF-PAR does not do this. Raw agent outputs frequently overlap, sometimes contradict each other, and always require synthesis before they can be presented as coherent advice. The Response Fusion Engine handles this in three steps.

Deduplication identifies semantically equivalent recommendations across agents. When both the Learning Agent and the Planning Agent independently recommend additional daily study time for a particular subject, their recommendations are not listed twice — they are merged into a single statement with the combined evidence of both agents behind it. Semantic equivalence is measured using cosine similarity on the embedding representations of the recommendation texts.

Conflict detection and resolution handles the harder case where agents disagree. Consider a scenario where the Planning Agent recommends 60 minutes per day for a subject based on schedule constraints, while the Risk Agent classifies that same subject as high-risk and the Learning Agent identifies three critical weak topics — together implying that 60 minutes is insufficient. The fusion engine detects this conflict and resolves it by prioritizing the recommendation with stronger evidence support: more student state fields corroborating it, higher inter-agent agreement, and a higher individual agent confidence score. The losing recommendation is not discarded — it is retained in the reasoning trace so the student can see that the system considered the tradeoff.

Evidence chain construction links every surviving recommendation to the specific student data that supports it:

```
Recommendation : "Allocate 90 min/day to Artificial Intelligence"
Evidence :
  · Internal mark     : 58%  (threshold: 65%)
  · Attendance        : 76%  (adequate — not a bottleneck)
  · Weak topics       : CNN, Backpropagation, RNN
  · Days to exam      : 32
  · Agent agreement   : Learning, Planning, Risk (3 of 3)
```

This evidence chain is stored in the RecommendationLog table, making every recommendation the system has ever produced queryable, auditable, and available for longitudinal analysis of student outcomes.

H. Confidence-Aware Explainable Decision Generation

The final layer of the pipeline addresses a problem that educational AI systems have historically struggled with: producing recommendations that students and faculty can actually trust. A recommendation without reasoning is an instruction. A recommendation with reasoning is advice. The distinction matters enormously in an educational context, where students need to understand *why* a particular course of action is being suggested in order to commit to it.

IDMAF-PAR computes a composite confidence score *C_final* for each recommendation:

$$C_final = w■ \cdot agent_confidence + w■ \cdot evidence_strength + w■ \cdot intent_match + w■ \cdot inter_agent_agreement$$

Each term captures a distinct dimension of recommendation quality. *agent_confidence* reflects how certain the contributing agents were in their individual outputs. *evidence_strength* measures the ratio of populated student state fields to the total fields required — a recommendation built on complete data is more trustworthy than one built on partial data. *intent_match* uses cosine similarity between the original query embedding and the recommendation embedding to verify that the output actually addresses what the student asked. *inter_agent_agreement* rewards recommendations that multiple independent agents arrived at without coordination — convergent reasoning across independent perspectives is a strong signal of validity.

The weights $w■$, $w■$, $w■$, $w■$ are empirically determined through held-out evaluation, with the constraint that they sum to 1.0.

The Explainability Layer formats the final output as a structured, human-readable recommendation:

```
FinalRecommendation {
  student         : "CSE — Semester 6"
  academic_risk   : "Medium"
  recommendations : [
    {
```

```
      subject     : "Artificial Intelligence",
      action      : "90 min/day for 30 days",
      confidence  : 91%,
      why         : [
        "Internal mark is 58% — below the 65% passing threshold",
        "Learning Agent identified 3 critical weak topics",
        "Risk Agent classified subject as Medium academic risk",
        "All 3 selected agents independently converged on this recommendation"
      ],
      target      : "Improve internal equivalent score from 58% to 75+"
    }
  ],
  next_review     : "7 days"
}
```

Two properties of this output are worth highlighting. First, the confidence score is not decorative — it is a computed value derived from four measurable dimensions of recommendation quality, and it changes meaningfully as student data changes. Second, the reasoning chain is not a post-hoc explanation appended to a black-box output. It is constructed during the fusion stage from actual evidence chains, which means it reflects what the system genuinely used to produce the recommendation.

This combination — quantified confidence, transparent evidence, and inter-agent corroboration — positions IDMAF-PAR not as a chatbot that dispenses advice, but as a decision-support system that helps students and faculty reason together about academic priorities. In contexts where student trust and institutional accountability both matter, this distinction is not cosmetic. It is the point.